

PetroCard Wallet — Frontend Integration

Sep 29, 2026 · @Akhil Sachan

What the wallet is

One common wallet that every active PetroCard spends from. You recharge the wallet, not the card.

If the wallet holds ₹5,000 and a fuel entry of ₹1,000 is made on any PetroCard, the wallet drops to ₹4,000 — regardless of which card was used.

Scope for this release is the **web app only**. The mobile app gets no wallet API and no new permission.

Read this before building any screen

The balance is not a stored number. Never compute it on the frontend.

The backend derives it on every read: total recharges minus every fuel entry currently holding money. There is no counter to keep in sync, which means the balance can change without the frontend having done anything — someone else approves a fuel entry, rejects one, or records a recharge.

Three rules follow:

1. After any action that could move money, re-read `GET /petro-card-wallet/balance`. Do not add or subtract locally.
2. The create, update and delete recharge responses each return the fresh `balance`, so a recharge action needs no second call.
3. A fuel entry that is rejected or deleted restores its amount automatically. There is no "restore" API to call, and nothing for the frontend to trigger.

Wallet APIs

All routes sit under `/api/v1/petro-card-wallet`.

Method	Path	Purpose
GET	<code>/balance</code>	Current balance + breakdown
GET	<code>/transactions</code>	Recharges and fuel deductions in one list

Method	Path	Purpose
GET	/recharges	Recharge-only list — the CRUD grid
POST	/recharges	Record a recharge
PATCH	/recharges/:id	Correct a recharge
DELETE	/recharges/:id	Delete a recharge

GET /balance

```
{
  "balance": 4000,
  "totalRecharged": 5000,
  "totalConsumed": 1000,
  "pendingConsumed": 1000,
  "approvedConsumed": 0
}
```

The breakdown is there so a low or negative balance can be explained on screen without a second call. `pendingConsumed` is money held by fuel entries still awaiting approval.

GET /transactions

Query: `page`, `pageSize`, `type` (`RECHARGE` or `FUEL`, omit for both), `dateFrom`, `dateTo`.
Newest first.

```
{
  "records": [
    { "type": "FUEL", "id": "<fuelExpenseId>", "date": "2026-09-28",
      "amount": -1000, "editable": false,
      "meta": { "cardNumber": "...", "vehicleNumber": "...", "employeeName": "...",
        "approvalStatus": "pending", "remarks": "..." } },
    { "type": "RECHARGE", "id": "<rechargeId>", "date": "2026-09-27",
      "amount": 5000, "editable": true,
      "meta": { "referenceNumber": "UTR...", "paymentMode": "NEFT",
        "paidFromAccount": "...", "remarks": "...", "recordedBy": "..." } }
  ],
  "totalRecords": 2
}
```

Fuel amounts come back **negative**, recharges positive — render the sign as given.

`editable` tells you which rows get the edit and delete buttons; `meta` differs by `type`.

There is deliberately no running "balance after" column. The list is filtered and paginated, so a running total would be computed over whatever subset was requested and would not match the real balance at that moment. Show the figure from `/balance` in the header instead.

POST /recharges

```
{ "amount": 5000, "rechargeDate": "2026-09-27T00:00:00.000Z",
  "referenceNumber": "UTR12345", "paymentMode": "NEFT",
  "paidFromAccountId": "<uuid>", "remarks": "..." }
```

Only `amount` and `rechargeDate` are required. `amount` must be greater than zero — a negative or zero value is a 400. There is no manual debit API by design; a mistake is fixed by editing or deleting the recharge.

Response: `{ "message": "...", "id": "<uuid>", "balance": 5000 }`.

PATCH /recharges/:id

Same fields, all optional. **Only the fields you send are changed** — sending just `{ "remarks": "..." }` leaves the amount alone. Returns the fresh `balance`.

DELETE /recharges/:id

Soft delete. The balance reverses by itself. Returns the fresh `balance`.

Screens to build

1. Wallet page

Three parts, one page:

- **Balance header** — from `GET /balance`. Show `balance` large. Consider showing `pendingConsumed` beside it as "held by pending entries", because that is usually what explains a balance lower than someone expects.
- **Transaction summary** — from `GET /transactions`. Mixed list, with a type filter and a date range. Fuel rows link out to the fuel expense screen; recharge rows carry the edit and delete buttons.
- **Recharge grid** — from `GET /recharges`, with add, edit and delete. `search` matches reference number or remarks.

Whether the summary and the grid are two tabs or one list with a filter is your call. The summary already covers the grid when `type=RECHARGE` is passed, so one list with a filter is enough if you prefer it.

2. Dashboard tile

`GET /dashboard/overview` now carries:

```
"petroCardWallet": { "balance": 4000 }
```

A single figure on the web dashboard. `GET /dashboard/mobile` is unchanged and carries no wallet field.

What changes on the fuel entry screen

`petro_card` is now a real payment mode

A fuel entry is treated as a PetroCard fill when `paymentMode` is `petro_card`. That value did not previously exist in the payment-modes config, so it could not be submitted at all — it has now been added.

Refetch `GET /configurations/details` for the fuel payment modes; `Petro Card` will appear in the list. Nothing special is needed to render it — it arrives like every other mode.

`transactionType` is not what marks a PetroCard fill. Keep sending whatever you send today.

The new 400: insufficient balance

Creating a PetroCard fuel entry for more than the wallet holds is now refused:

```
Insufficient PetroCard Wallet balance. Available ₹4,000.00, this entry needs ₹5,000.00. Recharge the wallet before recording this fuel entry.
```

The message already names both figures, so showing it as-is is fine. Two things worth doing anyway:

- Show the current balance on the fuel entry form when `petro_card` is selected, so the refusal is not a surprise at submit time.
- The same 400 applies to **force-create** and to **editing** an entry upward. Editing downward, or to another payment mode, always succeeds.

This gate applies to creation only. Approving or rejecting an entry is never blocked by the balance.

What moves the balance

Every row below is verified on dev. Refetch the balance after any action in the ✓ rows.

Action	Balance moves?	Notes
Fuel entry created (pending)	✓ deducts	Pending already holds the money
Pending → approved	no change	Both states hold it, so approval moves nothing
Approved → rejected	✓ restores	This is the "rejected restores" requirement
Rejected → approved	✓ deducts again	The status can flip either way
Entry cancelled	✓ restores	Treated like rejected
Fuel amount edited up or down	✓ by the difference only	₹2,000 edited to ₹3,000 moves ₹1,000
Payment mode changed off <code>petro_card</code>	✓ restores in full	And onto it, deducts in full
Fuel entry deleted	✓ restores	
Cash / UPI / credit entry, any action	no change	These never touch the wallet
Recharge created	✓ adds	Response carries the new balance
Recharge amount edited	✓ by the difference	
Recharge deleted	✓ reverses in full	

The practical consequence: the balance shown on one screen can go stale because of something someone did on another. Refetching `/balance` when the wallet page or dashboard regains focus is worth doing.

Permissions

Two new permissions, both in the existing `petro_card` module, both web-only.

Permission	Gates
<code>petro-card.wallet-view</code>	Wallet page, balance header, transaction summary, recharge list, dashboard tile

Permission	Gates
<code>petro-card.wallet-manage</code>	Add, edit and delete recharge buttons

They are split because seeing the balance and putting company money into the wallet are different authorities — plenty of people should see one and not do the other.

Granted to **SUPER_ADMIN**, **ADMIN** and **OPERATION_MANAGER**, matching who already holds `petro-card.add`. Anyone else gets a 403 on every wallet route, so hide the whole wallet page rather than rendering an empty one.

Edge cases to design for

A negative balance is valid and will happen. Deleting or reducing a recharge is allowed even when the money has already been spent — the operator is trusted to correct their own records. Do not clamp the figure at zero and do not treat it as an error. Render it clearly (red, minus sign) with a prompt to recharge; it is the signal that a top-up is overdue.

While the balance is at or below zero, every new PetroCard fuel entry is refused. That is the deliberate consequence of allowing the delete. The fuel form should make this visible rather than letting a driver fill in the whole form and fail at submit.

A fresh wallet reads zero, not empty. `GET /balance` returns `0` with no recharges recorded. The transaction list comes back with `records: []`. Design an empty state for the list, but the balance header always has a number.

The balance can move while a screen is open. Another user approving or rejecting a fuel entry changes it. Nothing pushes this to the frontend, so a refetch on focus is the simplest answer.

Exactly the balance is allowed. A ₹6,000 entry against a ₹6,000 balance succeeds and leaves zero. Only ₹1 more is refused. Use `>` and not `>=` if you mirror the check client-side for a warning.

Status, scope and open questions

Status. Built and verified on the dev database — 61 assertions covering every row of the table above, all passing. Three migrations have run on **dev only**. UAT and production are untouched, so these APIs are not live there yet.

Out of scope, deliberately:

- The mobile app. No wallet API, no new permission, no change to `GET /dashboard/mobile`. The three existing `mobile_card_wallet` permissions are UI visibility flags that back nothing; they were left alone.
- Per-card balances. There is one wallet; cards carry no balance of their own.
- A manual debit API. Money only leaves the wallet through a fuel entry.

Open questions back to us:

1. Should the fuel entry form call `/balance` on load, or only when `petro_card` is picked? The second is fewer calls; the first lets you warn earlier.
2. Do you want a low-balance threshold (a warning below, say, ₹5,000)? Nothing like that exists today — it would be a small backend addition if the UI wants it.
3. Approving a previously-rejected entry can push the balance negative, because the money comes back out. We chose not to block the approver mid-flow. Say if you would rather that were refused.